

Redpanda: FDE Interest - Takehome

1 message

Abel Mathew <abel@redpanda.com>
To: daniel.takabayashi@gmail.com
Cc: Eve Hudak <eve.hudak@redpanda.com>, Patrick Angeles <pa@redpanda.com>

Hi Daniel,

Thanks again for your time on Friday. The team enjoyed the presentation and is excited about the prospect of potentially working together.

I was told you are interested in the FDE positions we have open here at Redpanda. That's exciting and your profile definitely looks like a fit. That said, part of what we consider when evaluating development experience. It looks like your Apache Marvin work provides that experience. To dig one level deeper, I'd like to ask you to complete our FDE takehome exercise. I've pasted it at the bottom of this email. Please read through it carefully and complete it at your earliest convenience. The hope is that it doesn't take more than a couple of hours, but please let me know if you hit any major hurdles or if you need any help. We'll forgo the 60 minute walkthrough discussion since you've already done the team presentation. I'll read your write up on design decisions and send over any questions I may have afterwards. Thanks and I look forward to seeing what you come up with.

Abel

Field Deployed Engineer — Build Exercise

- Build a small system that pulls public data, transforms it, runs an agent or reasoning loop over each item, and serves the result.
- What to build**
- 1. **Ingest** a public data source (no signup required). Pick from the list below or bring your own.
 - 2. **Transform** in [Redpanda Connect](#) — filter, project, route.
 - 3. **Reason** over each item with an LLM. More than one prompt: routing, multi-step extraction, retries on bad output, confidence-based branching — pick what you want.
 - 4. **Serve** the result somewhere we can see it. A web page, a JSON endpoint, or a CLI that reads from the store. Postgres or a Redpanda topic (or both).
 - 5. **Run locally** with `docker compose up`.
- Stack (all free, all local, all open source)**

Piece	Image	Notes
Broker	docker.redpanda.com/redpandadata/redpanda	Use <code>--mode=dev-container</code> for single-node.
Pipeline	docker.redpanda.com/redpandadata/connect	Apache 2.0. docs
LLM, local	ollama/ollama + any open-weights model	Use <code>ollama_chat</code> or point <code>openai_chat_completion</code> 's <code>server_address</code> at it.
LLM, hosted	OpenAI / Anthropic / Bedrock — your call	Include a <code>.env.example</code> .
Storage	postgres:16	Or just a topic if you prefer.

Off-limits: Redpanda Cloud (signup), Enterprise Connect connectors (CDC, Snowflake Streaming, Iceberg, Salesforce — see [licensing](#)), a2a_message (Cloud OAuth2).

Pick a data source

Source	URL	Shape
Wikipedia recent-changes	https://stream.wikimedia.org/v2/stream/recentchange	SSE firehose, ~50/sec, no auth. Set a User-Agent header (Wikipedia)
Hacker News Firebase	https://hacker-news.firebaseio.com/v0/	Poll-based. <code>/newstories.json</code> returns IDs; secondary fetch for details
USGS earthquakes	https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_hour.geojson	GeoJSON, polled. Time is epoch-ms , not seconds. Same items repeated
GitHub public events	https://api.github.com/events	PRs/issues/pushes. 60 req/hr anonymous — set <code>GITHUB_TOKEN</code>
Your own	Bluesky firehose, GDELT, NOAA, Reddit JSON, OpenSky	Bring it.

- Example agent ideas (illustrative — invent your own if you'd rather):
- Wikipedia: classify edits `vandalism` | `substantive` | `trivia` | `unclear`; or summarize what changed across the last N edits per article.
 - HN: route new stories into your own categories with confidence; or compare headline to article body and flag mismatches.
 - USGS: decide `notify` | `monitor` | `ignore` from magnitude + depth + place; or write a regional brief using prior events in the same area.
 - GitHub: triage opened PRs into `security` | `feature` | `refactor` | `docs` | `dependency-bump` with affected area.

Hints (so you spend the time on judgment, not trivia)

Things we hit ourselves while writing the reference solution. Not stumpers — we don't want you to rediscover these.

- **Filter before the model.** Firehoses are noisy. Drop bots, heartbeats, irrelevant namespaces *first*. Calling the model on every record is the wrong shape.
- **SSE in Connect** is `http_client` with `stream.enabled: true` + `lines_scanner`. Strip the `data: prefix` in Bloblang. Use `.catch(deleted())` on `doesn't fail-closed` inside `if`, so a single non-JSON heartbeat will pass through as a raw string and break everything downstream.
- **Bloblang mapping rebuilds root from scratch.** A trailing mapping that sets one field will drop everything else. Start with `root = this` or fold field in (Inside `branch.result_map` the rule inverts — `branch` grafts your assignments back in, so partial `root.X = ...` is correct there.)
- **Use branch for LLM calls.** `request_map` builds the prompt; `result_map` grafts the model output back onto the original message. Don't overwrite model response.
- **Small/local models produce dirty JSON** — extract the first `{...}` block, fall back to a default on parse failure, and **normalize labels** (`.string().lowercase()`). Models drift outside your enum.
- **First-poll cold-start LLM failures pass through branch silently** — rows land unclassified. Pair with an UPSERT sink so a later poll fixes them.
- **For poll-based sources, dedupe** with a Connect `cache` resource keyed on item id, or `upsert`. Without one of these you'll either reprocess or PK-contradict.
- **Public APIs need a User-Agent.** Wikipedia 403s without one; GitHub rejects empty UAs.
- **Postgres TIMESTAMPTZ rejects raw epoch numbers** — format to an ISO string in Bloblang first.

Required write-up (~30 min)

In your `README.md`, include a **Tradeoffs** section (½ to 1 page). Pick **two** of these pairs — whichever are most relevant to what you actually built. One chose, the alternative, when you'd flip. Cite anything that shaped your thinking.

- Connect as the sink vs. app-side writes from a topic
- `branch` vs. `mutation` for LLM enrichment
- One topic with a label column vs. topic-per-label routing
- One classification call vs. a multi-step reasoning loop
- Synchronous LLM inside the pipeline vs. async worker reading from a topic
- Connect AI processor vs. a sidecar service you wrote yourself

Also include a short paragraph on **what surprised you** and **where this would break in production**.

Deliverable

A git repo (zip or link):

- `README.md` — copy-pasteable run instructions, the Tradeoffs section, surprises, production-failure notes
- `docker-compose.yml` — everything needed to run, including any model proxy
- Connect configs, app code, schema, seeds
- `.env.example` if you use a hosted LLM

We will run it. Cleanly starting is part of the signal.

Walkthrough (60 min, follow-up session)

Screen-share, bring it up live, walk us through. We'll ask: why this source, why this transform shape, why this agent topology, where the failure mode for a real customer. AI tools and any reference material allowed throughout — we care about the result and that you can explain every choice.

What we are evaluating

Dimension	What "strong" looks like
Connector fluency	Picks a sensible Connect input/output. Config is clean. Understands batching, retries, error paths — not just the happy path.
Data sense	Transformation makes the data more useful, not just reshaped. Schema choices are deliberate. Doesn't pipe junk into the model.
Agent design	Reasoning loop has structure — not one giant prompt. Handles model failure, malformed output, cost. Knows when to use a model and when not to.
Output usability	Result is something a person could actually look at and act on. UI/endpoint is plain but works. Latency and freshness make sense.
Local repro	One command brings it up. No missing files, no manual steps the README forgot. Logs are useful.
Write-up and walkthrough	README is short and honest. Candidate can explain every choice and name the cuts. Talks about failure modes without being prompted.
Architectural reasoning	Tradeoffs section is real — names the alternative, names when they'd flip the decision, cites what shaped their thinking. Not boilerplate.